

Python 版智能决策系统开发清单与多 Agent 架构设计

第一部分：十二块开发清单（2026.09 执行版）

第一块：环境准备（Python 适配）

- ☐ Python 3.12+（推荐 3.12，性能与类型注解支持最佳）
- ☐ IDE: PyCharm Professional 或 VS Code (Python Extension + Pylance)
- ☐ 包管理: uv (推荐, 极速) 或 Poetry / pip-tools
- ☐ Git & Docker Desktop
- ☐ 数据库: **MySQL 8.0**、Redis 7
- ☐ API 调试: Postman 或 Bruno
- ☐ 虚拟环境: .venv 隔离, 禁止全局安装依赖

第二块：数据库四张表（ORM 适配）

- ☐ 原始数据表 (raw_data)
- ☐ 分析结果表 (analysis_results)
- ☐ 决策结果表 (decision_results)
- ☐ 规则节点表 (rule_nodes): 字段包含 parent_id, condition_type, threshold, operator, weight, action

注：使用 SQLAlchemy 2.0 Async ORM 定义模型，配合 Alembic 做版本迁移。

第三块：后端四层架构（Python 标准分层）

- ☐ api/ (Controller 层): 仅做参数校验(Pydantic)与响应组装, 不含业务逻辑
- ☐ services/ (Service 层): 核心业务规则、事务控制
- ☐ agents/ (Agent 层): LangGraph/LangChain 编排、Prompt 模板、工具函数
- ☐ repositories/ (DAO 层): 封装所有 DB/Redis 操作, 禁止 SQL 裸写
- ☐ core/ (Common 包): 配置管理、异常定义、依赖注入、日志、安全中间件

第四块：八块基础代码（Python 等价实现）

- ☐ Pydantic Models: 替代集合泛型, 严格定义输入输出 Schema
- ☐ FastAPI Router: RESTful 接口定义, 自动 OpenAPI 文档
- ☐ Service 业务逻辑: 异步函数 (async def), 依赖注入获取 Session
- ☐ SQLAlchemy Async Mapper: 异步数据库访问层
- ☐ Redis 缓存: redis.asyncio + 装饰器封装缓存策略
- ☐ 全局异常处理: 自定义 AppException + FastAPI Exception Handler
- ☐ JWT 认证中间件: python-jose + Security Scheme 依赖
- ☐ Loguru/Structlog 配置: 结构化日志, 替代 Logback, 支持 JSON 输出

第五块：三个 Agent（LangGraph 状态机实现）

- ☐ 侦察兵 Agent: 读取原始数据, 调用 get_raw_data Tool, 输出结构化事实
- ☐ 分析师 Agent: 基于事实计算风险评分, 支持 ReAct 推理循环
- ☐ 决策官 Agent: 递归遍历决策树 (LangGraph Conditional Edge), 综合打分并生成决策报告

注：使用 LangGraph 构建有向图，避免纯 Chain 的线性局限。详见第三部分多 Agent 架构设计。

第六块：业务层（异步消息队列）

- ☐ RuleEngine: Python 实现规则匹配引擎（可复用 rule-engine 库或自研 DSL）

- ☐ RabbitMQ 生产者: aio-pika 异步推送预警消息
- ☐ Celery/arq 任务队列: 耗时分析任务异步化, 避免阻塞主线程

第七块: 前端核心 (保持不变, 接口对接调整)

- ☐ 总览大屏: ECharts 雷达图
- ☐ 详情页: 订单 + 物流数据展示
- ☐ WebSocket: 对接 FastAPI WebSocket Endpoint 接收实时预警

注: 后端提供 SSE/WebSocket 双协议支持流式 Agent 响应。

第八块: 测试 (Python 测试体系)

- ☐ pytest + pytest-asyncio: 每个 Agent 编写单元测试, Mock LLM 调用
- ☐ httpx.AsyncClient: 集成测试, 跑通完整链路
- ☐ Postman/Bruno: 手动验收关键接口
- ☐ 覆盖率: pytest-cov 确保核心逻辑 >80%

第九块: 部署 (容器化)

- ☐ Dockerfile: 多阶段构建, slim 镜像, 非 root 用户
- ☐ docker-compose.yml: 编排 App + MySQL 8.0 + Redis + RabbitMQ + Nginx
- ☐ Nginx 反向代理: HTTPS + WebSocket 升级头配置
- ☐ 备份脚本: mysqldump 全量 + binlog 增量备份, 定时任务挂载

第十块: 安全 (Python 专项)

- ☐ JWT 校验: 密钥从环境变量读取, 禁止硬编码
- ☐ SQL 注入防护: 全程 ORM 参数化查询, 禁止 f-string 拼接 SQL
- ☐ 敏感字段脱敏: Pydantic Field Serializer 自动脱敏
- ☐ 容器安全: 非 root 运行 + 只读文件系统
- ☐ 依赖扫描: pip-audit 或 safety 检查开源组件漏洞

第十一块: 可观测性

- ☐ traceId 贯穿: OpenTelemetry SDK + FastAPI Middleware 自动注入
- ☐ 关键决策点日志: Agent 每一步推理/工具调用记录结构化日志
- ☐ Health Check: /health 端点检查 DB/Redis/MQ 连通性
- ☐ LangSmith/LangFuse: Agent 调用链追踪与评估 (强烈推荐)

第十二块: 报价和投标 (人天重估)

- ☐ 报价清单: 按 Python 全栈 + AI Agent 工程师费率重新核算 (Python 开发效率通常高于 Java, 但 Agent 调试成本增加, 建议维持 45 人天或微调结构)
 - ☐ 投标书技术部分:
 - 架构图更新为 FastAPI + LangGraph + MySQL 8.0
 - 甘特图增加” Agent Prompt 调优” 与” 评估集构建” 阶段
 - 团队介绍突出 Python AI 工程化经验
-

第二部分：Java 到 Python 关键变更对照表

原 Java 项	Python 替代方案	注意事项
Spring Boot	FastAPI + Uvicorn	异步优先，注意事件循环阻塞
MyBatis	SQLAlchemy 2.0 Async	学习曲线较陡，务必用 async session
Spring Security	python-jose + FastAPI Depends	无声明式权限，需手写依赖
Logback	Loguru / Structlog	配置更简单，但需手动对接 ELK
JUnit	pytest + pytest-asyncio	fixture 机制不同于 @Before/@After
Maven/Gradle	uv / Poetry	锁定文件 uv.lock / poetry.lock 必提交
LangChain4j	LangChain / LangGraph	Python 版生态更全，但版本迭代快，锁版本
MySQL	MySQL 8.0	使用 aiomysql 或 SQLAlchemy async 驱动，注意字符集 utf8mb4

第三部分：多 Agent 协同架构设计（Python/LangGraph 版）

1. 角色定义与能力边界（遵循最小依赖原则）

Agent 角色	核心职责	输入	输出	对应 LangGraph 节点
侦察兵 (Scout)	数据采集与清洗	原始订单/物流 ID	结构化事实 JSON	scout_node
分析师 (Analyst)	风险评分与异常检测	结构化事实 + 历史规则	风险分值 + 异常标签	analyst_node
决策官 (Decider)	递归遍历决策树、综合裁决	风险分值 + 业务上下文	最终决策 + 置信度 + 解释	decider_node
协调器 (Orchestrator)	状态路由、重试、人机介入	全局 State	下一跳节点指令	router_edge (条件边)

关键原则：每个 Agent 只关注自身输入输出，不直接调用其他 Agent。所有交互通过 Shared State（共享状态）完成，避免循环依赖。

2. 通信与协作机制选型

根据业务特性（规则驱动 + LLM 推理混合），推荐采用混合协作模式：

- 状态传递：使用 LangGraph 的 TypedDict 定义全局 State Schema，替代传统消息队列传参。

- 任务分配：采用层次化分配 + 条件路由。Orchestrator 根据 Scout 返回的数据完整性，动态决定是进入 Analyst 还是触发“数据补全”子流程。
- 共识机制：对于高风险决策，引入 Self-Reflection（自反思）或 Critic Agent 进行二次校验，而非简单投票。

LangGraph State 定义与条件路由示例：

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, END

class DecisionState(TypedDict):
    raw_data_id: str
    structured_facts: dict | None
    risk_score: float | None
    decision_result: dict | None
    retry_count: int

def should_continue(state: DecisionState) -> str:
    # 协调器：根据状态决定下一步
    if state["structured_facts"] is None:
        return "scout"
    if state["risk_score"] is None:
        return "analyst"
    if state["retry_count"] > 3:
        return "human_review" # 容错：超限转人工
    return "decider"

# 构建图
workflow = StateGraph(DecisionState)
workflow.add_node("scout", scout_agent)
workflow.add_node("analyst", analyst_agent)
workflow.add_node("decider", decider_agent)
workflow.add_conditional_edges("router", should_continue, {
    "scout": "scout",
    "analyst": "analyst",
    "decider": "decider",
    "human_review": END
})
```

3. 与现有清单的集成点

- 第四块基础代码：Agent 层需新增 graphs/ 目录存放 LangGraph 定义，tools/ 目录存放各 Agent 专用工具函数。
- 第六块业务层：RuleEngine 作为 Tool 注入 Analyst Agent，而非独立服务。决策结果由 Decider Agent 写入 DB 后，再异步推送到 RabbitMQ。
- 第十一块可观测性：必须集成 LangFuse / LangSmith。LangGraph 原生支持 OpenTelemetry，可在 trace 中清晰看到每个节点的输入输出、Token 消耗及耗时。
- 第八块测试：Agent 单元测试使用 langgraph.testing 框架，Mock LLM 响应，验证状态流转逻辑而非 Prompt 内容。

4. 避坑指南（2026 实战经验）

- 不要过度设计：三个 Agent 足够时，不要强行拆成五个。复杂度应随业务增长渐进式增加。

- State 要轻量：不要在 State 中存大对象（如完整日志、图片）。只存 ID 或摘要，需要时通过 Tool 实时查询。
- Prompt 版本化：将 Prompt 模板从代码中剥离，存入 DB 或配置文件，支持热更新而不重启服务。
- 设置超时与熔断：LLM 调用可能卡死。为每个 Agent 节点设置 timeout=30s，失败后走降级路径（如返回默认低风险评分）。
- 评估集先行：在写 Agent 代码前，先准备 50+ 条标注好的”输入-期望输出”测试用例。没有评估集的 Agent 开发等于盲调。

5. 推荐学习资源

- LangGraph 官方文档：重点看 StateGraph、Conditional Edges、Persistence 章节。
- 《Multi-Agent 多智能体完整详解》：系统理解六大核心要素与协作模式选型。
- Azure Multi-Agent Workflow 架构指南：虽为 Azure 体系，但其顺序/并发/群组聊天/移交四种协调模式的抽象具有普适性。

本文档整合了全部十二块开发清单、Java 到 Python 技术栈变更对照表、以及多 Agent 协同架构设计方案，可直接作为项目执行蓝图使用。数据库统一使用 MySQL 8.0。